

موانع یک لاما

یک لاما می‌خواهد از منطقه‌ای در فلات «آند» عبور کند. او نقشه‌ای از منطقه به شکل یک جدول $N \times M$ دارد. سطرهای نقشه از بالا به پایین از 0 تا $N - 1$ و ستون‌ها از چپ به راست از 0 تا $M - 1$ شماره‌گذاری شده‌اند. خانه‌ی واقع در سطر i و ستون j ($0 \leq i < N, 0 \leq j < M$) با (i, j) نشان داده شده است.

لاما آب و هوای منطقه را مطالعه کرده و کشف کرده است که تمام خانه‌های هر سطر نقشه دارای دمای یکسان و تمام خانه‌های هر ستون نقشه دارای **رطوبت** یکسان هستند. لاما دو آرایه شامل اعداد صحیح با نام‌های T و H به ترتیب به طول N و M به شما داده است. $T[i]$ (به ازای $0 \leq i < N$) دمای خانه‌های سطر i و $H[j]$ (به ازای $0 \leq j < M$) رطوبت خانه‌های ستون j را نشان می‌دهد.

لاما همچنین گیاهان فلات را مطالعه کرده و متوجه شده است که خانه‌ی (i, j) **عاری از پوشش گیاهی** است اگر و تنها اگر دمای آن بیشتر از رطوبت آن باشد، یعنی $T[i] > H[j]$.

لاما فقط با دنبال کردن **مسیرهای معتبر** می‌تواند در فلات سفر کند. یک مسیر معتبر، دنباله‌ای از خانه‌های مجزا است که شرایط زیر را برآورده می‌کنند:

- هر جفت از خانه‌های متوالی در مسیر، یک ضلع مشترک دارند.
- تمام خانه‌های موجود در مسیر عاری از پوشش گیاهی هستند.

وظیفه شما پاسخ دادن به Q پرسمان است. برای هر پرسمان، چهار عدد صحیح به شما داده می‌شود: L, R, S و D . شما باید تعیین کنید که آیا یک مسیر معتبر وجود دارد که:

- مسیر از خانه‌ی $(0, S)$ شروع شود و به خانه‌ی $(0, D)$ ختم شود.
- تمام خانه‌های موجود در مسیر، بین ستون‌های L تا R (شامل هر دو ستون) قرار گیرند.

تضمین می‌شود که هر دو خانه‌ی $(0, S)$ و $(0, D)$ عاری از پوشش گیاهی باشند.

جزئیات پیاده‌سازی

اولین تابعی که باید پیاده‌سازی کنید به صورت زیر است:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : آرایه‌ای به طول N که دمای هر سطر را مشخص می‌کند.
- H : آرایه‌ای به طول M که رطوبت را در هر ستون مشخص می‌کند.
- این تابع دقیقاً یک بار برای هر تست کیس، قبل از هرگونه فراخوانی `can_reach` فراخوانی می‌شود.

دومین تابعی که باید پیاده‌سازی کنید به صورت زیر است:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : اعداد صحیحی که یک پرسمان را توصیف می‌کنند.
- این تابع Q باربرای هر تست کیس فراخوانی می‌شود.

این تابع باید مقدار `true` را برگرداند اگر و تنها اگر یک مسیر معتبر از خانه $(0, S)$ به خانه $(0, D)$ وجود داشته باشد، به طوری که تمام خانه‌های مسیر، بین ستون‌های L تا R (شامل هر دو) قرار گیرند.

محدودیت‌ها

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ برای هر i که $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ برای هر j که $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- هر دو خانه‌ی $(0, S)$ و $(0, D)$ عاری از پوشش گیاهی هستند.

زیرمسئله‌ها

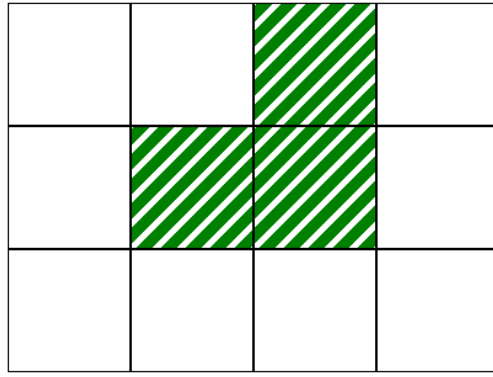
زیرمسئله	امتیاز	محدودیت‌های اضافی
۱	10	برای هر پرسمان $L = 0, R = M - 1, N = 1$.
۲	14	برای هر پرسمان $L = 0, R = M - 1, 1 \leq i < N$ که $T[i - 1] \leq T[i]$.
۳	13	برای هر پرسمان $L = 0, R = M - 1, N = 3$ و $T = [2, 1, 3]$.
۴	21	برای هر پرسمان $L = 0, R = M - 1, Q \leq 10$.
۵	25	برای هر پرسمان $L = 0, R = M - 1$.
۶	17	بدون هیچ محدودیت اضافی.

مثال

فراخوانی زیر را در نظر بگیرید:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

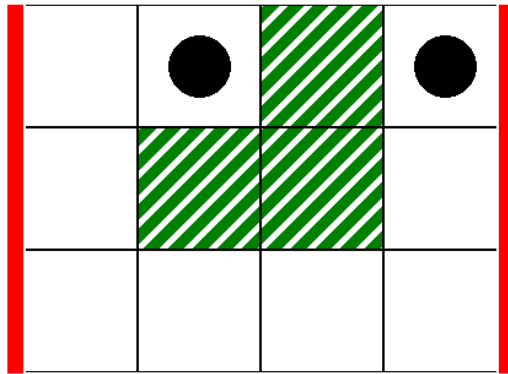
این فراخوانی مربوط به نقشه‌ی زیر است که در آن خانه‌های سفید عاری از پوشش گیاهی هستند:



به عنوان اولین پرسمان، فراخوانی زیر را در نظر بگیرید:

```
can_reach(0, 3, 1, 3)
```

این فراخوانی مربوط به وضعیت تصویر زیر است، که در آن خطوط عمودی ضخیم، محدوده ستون‌ها از $L = 0$ تا $R = 3$ را نشان می‌دهند و دیسک‌های سیاه، خانه‌های شروع و پایان را نشان می‌دهند:



در این حالت، لاما می‌تواند از طریق مسیر معتبر زیر از خانه $(0, 1)$ به خانه $(0, 3)$ برسد:

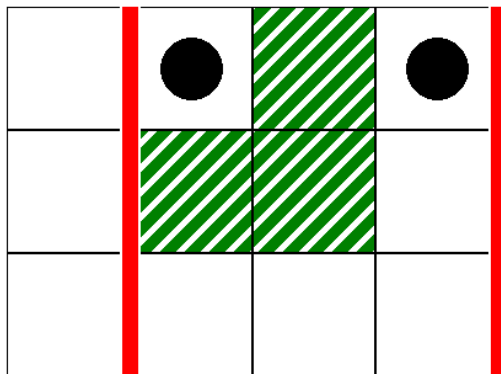
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

بنابراین، این فراخوانی باید `true` برگرداند.

به عنوان پرسمان دوم، فراخوانی زیر را در نظر بگیرید:

```
can_reach(1, 3, 1, 3)
```

این مربوط به سناریوی تصویر زیر است:



در این حالت، هیچ مسیر معتبری از خانه $(0, 1)$ به خانه $(0, 3)$ وجود ندارد، به طوری که تمام خانه‌های موجود در مسیر، بین ستون‌های 1 تا 3 قرار گیرند. بنابراین، این فراخوانی باید `false` برگرداند.

گریدر نمونه

فرمت ورودی:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

در این جا $L[k], R[k], S[k]$ و $D[k]$ ($0 \leq k < Q$) پارامترهای هر فراخوانی `can_reach` را مشخص می‌کنند.

فرمت خروجی:

```
A[0]
A[1]
...
A[Q-1]
```

در این جا $A[k]$ ($0 \leq k < Q$) برابر 1 است اگر فراخوانی `can_reach(L[k], R[k], S[k], D[k])` مقدار `true` برگرداند، در غیر این صورت 0 است.